

Baseline 单 Path0 的低利用率反例

SSD 直通 HBM · 从输入计算到排队因果的新手教程

qos_storage_sim 可复现实验 · 修订版 v2

2026-09-05

这是一份**仿真反例教程**，不是 GLM-5.1 实机性能预测。

研究条件：4 个独立 NPU 执行通道、1 个 SSU、每请求 8 层、跨请求预取开启；Prefill 的 KV 数据从 SSD 直接传入 HBM，**不使用主机 DRAM 缓存或中转**。

在这组输入和当前 Baseline 模型中，四卡名义带宽和为 **39.8862 GiB/s**，中间 1 秒的平均计算时间利用率为 **76.2261%**；NPU0 仅为 **21.6560%**。

最重要的结论不是“平均带宽不够”，而是：

一层数据能否按时到达 HBM，取决于排队、服务与接收的总时间。平均带宽够用，不代表短计算层的读取 deadline 能被满足。等待会改变下一层的读取时刻，但不保证消除后续碰撞。

本版修订重点：

- 明确 SSD→HBM 数据路径，不引入 DRAM 缓存作为缓解手段。
- 拆开输入表，手算 NPU0 与 NPU1 的每个关键数字。
- 用完整时序纠正“等待期间读自己的数据，所以与此前排队无关”的归因。
- 补入五组敏感性测试、逐层 deadline 指标及 HBM 预取成本。
- 区分已验证事实、模型假设、现实预测和优化建议。

阅读路线：第 1-3 节建立概念；第 4 节照着算输入；第 5-7 节看 trace 与对照；第 8-10 节判断现实适用性和改进方向；第 11 节复现与核对来源。

目录

1 固定讨论范围：SSD 直通 HBM	3
2 先看结果，再审计测量口径	4
3 三个基本量：数据量、计算时间、读取时间	5
4 输入怎样算出来，以及怎样找到候选	6
5 为什么等待造成错位，仍没消除低利用率	10
6 归因看完整时序，不能只看等待时 SSD 在读谁	12
7 对照与敏感性：不只是一组精确周期	14
8 现实中的 SSD→HBM Prefill 会出现吗	15
9 写一个判断 Path 是否赶得上 deadline 的函数	16
10 无 DRAM 前提下怎样优化系统	18
11 复现、证据与来源	19

1 固定讨论范围：SSD 直通 HBM

1.1 数据路径与调度路径是两件事

按用户指定的场景：Prefill 所需的历史 KV 默认位于 SSD，读取后直接进入目标 NPU 的 HBM；不经过主机 DRAM 的数据缓存或 bounce buffer（中转缓冲区）。当前模拟器的相关阶段可以理解为：

命令进入 Path0 → SSU/SSD 服务 → 接收链路传入 HBM → 层数据 ready.

它没有主机 DRAM 数据缓存阶段，因此这次澄清**不要求改变已有仿真结果**。但模型也没有实现真实 HBM 容量分配、淘汰与 DMA 引擎争用；“没有 DRAM 阶段”不等于“已验证真实直通硬件”。

这里的“无 DRAM”限定的是 **KV 数据载荷**，不要求 CPU 完全不参与提交，也不要求控制面元数据消失。作为同类技术实例，NVIDIA GDS 允许存储与 GPU 内存直接 DMA，控制路径仍可由 CPU 驱动执行。这说明直通路径具有工程可行性，不证明本项目 NPU 使用同一实现。[S1：GDS 官方概述](#)

问题	本文采用的条件
数据经过哪里？	SSD 直接到 HBM，无主机 DRAM 数据缓存/中转
命令按什么顺序服务？	Baseline 全进 Path0，Path 内严格 FIFO
后端同时处理多少命令？	模型中每 SSU 只有一条活动命令

SSU 的共享服务带宽为 40 GiB/s，每 NPU 有独立的 50 GiB/s 接收链路；后者不是 HBM 本地读写带宽。

第一条不能推出后两条。直通只改变数据传输路径；队列是否可重排、介质是否并行、是否限制队列深度，必须另行确认。

1.2 四个 NPU 在模型里代表什么

每个 NPU 独立运行自己的固定请求流，batch_size=1。该请求等 I/O 时，模拟器不会调度另一个请求的计算去填满同一张卡；跨请求预取只帮助读取，不等于跨请求计算并发。

若真实“四张卡”共同组成一个张量并行或流水线并行组，而不是四个独立执行通道，就不能直接套用这四条独立时间线。必须重新确定每卡的计算、KV 分片、同步等待与链路负载。

1.3 三个结论边界

单位：后文按用户后续明确的 **40 GiB/s** 讨论。主输入 39.886158 GiB/s 等于 42.827436 GB/s，不满足字面的十进制 40 GB/s。原结果另有输入 37.172558 GiB/s = 39.913730 GB/s，平均利用率为 77.7745%。该审计只收紧输入需求，物理 SSU 仍为 40 GiB/s，不能说验证了“40 GB/s 的物理 SSU”。

搜索范围：76.2261% 是固定其它三条流后，NPU0 的整数 NQL=153...192 共 40 个候选中的最低值，不是全局最小值。

数据来源：data 有 84 个参数点，但没有概率、频率权重或测量设备说明。本例保留其参数关系并插值/外推，不能声称复现了真实请求分布。

2 先看结果，再审计测量口径

2.1 只统计中间一秒

每个 NPU 先完成至少 8 个请求，再稳定运行 500 ms，随后测量精确的 1,000 ms。主实验的绝对窗口为：

$$[2430.127253, 3430.127253] \text{ ms.}$$

利用率定义为**计算事件占据的时间比例**：

$$U_i = \frac{\text{窗口内 compute 覆盖时间}}{1000 \text{ ms}}, \quad \bar{U} = \frac{U_0 + U_1 + U_2 + U_3}{4}.$$

它不是峰值 FLOPS 利用率，也不是 MFU。显示 100% 不表示算子达到了硬件峰值吞吐。

NPU	计算时间 (ms)	等 I/O 时间 (ms)	利用率
0	216.560	783.440	21.6560%
1	832.484	167.516	83.2484%
2	1000.000	0.000	100.0000%
3	1000.000	0.000	100.0000%

平均为 **76.2261%**。每卡均有 active request，compute 与 I/O barrier（因数据未到齐而等待）覆盖完整一秒。不是因为 NPU0 没有任务才利用率低。

SSD busy 为 **869.105 ms**，实际服务吞吐为 **34.7642 GiB/s**。所以不是 SSD 始终满载；反复出现的是“突发提交—排队—计算停顿”的结构。

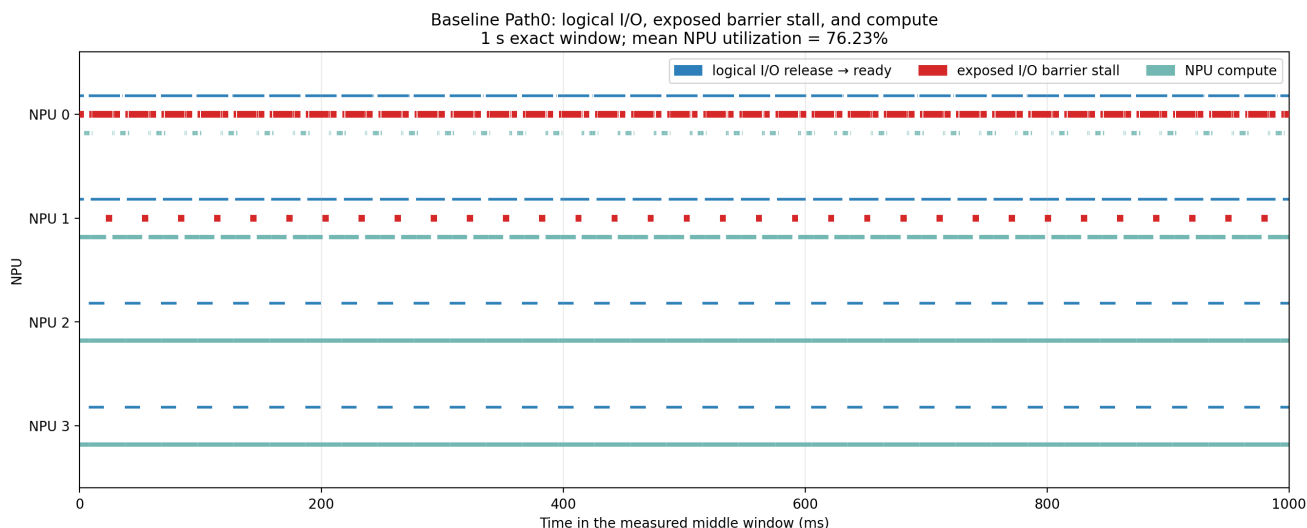


图 1：完整一秒的逻辑 I/O、等待与计算；红色表示有任务但数据未到齐。

2.2 中间窗口不能忽略所有 Layer 0

中间窗口排除了仿真最初的冷启动，但每运行 8 层就进入下一请求的 Layer 0。跨请求预取改变这些 Layer 0 的读取时刻，它们仍参与排队。

SLO 请求样本数是 47、8、4、4：这些是在窗口内接纳并标记、随后追踪到完成的请求，**不能说它们全部在这一秒内完成**。利用率只截取指定一秒，不计窗口外的计算时间。

3 三个基本量：数据量、计算时间、读取时间

3.1 token、NQL 与每层 KV

设总长度为 T ，本次新增 query token 数为 N (NQL)。本例把 $T - N$ 个历史 token 的 KV 作为 SSD 上待读取的前缀，每 token、每层采用 **1,408 byte** 的数据量模型。

$$V_{\text{bytes}} = (T - N) \times 1408, \quad V_{\text{GiB}} = V_{\text{bytes}} / 2^{30}.$$

不是每层重新计算全部历史 token；它们提供当前 query 所需的 KV。每层计算时间 C 来自 data 或其插值/外推，**不是只按 NQL 比例缩放**。

还要区分完整冷 Prefill 与读取已有前缀 KV 后处理新增 token：本例研究后者。若从未存在可读取的前缀 KV，就不能直接假设有同样的 SSD 读取量。

3.2 名义带宽是速率，不是读取时间

若 C 用 ms 表示，则：

$$B_i = \frac{V_i}{C_i / 1000} \quad (\text{GiB/s}).$$

含义：在无 I/O 等待的连续计算节拍下，要用本层计算时间掩盖下一层读取，需要多少平均速率。它不是瞬时提交速率、带宽预留或发生阻塞后的实际吞吐。

结果字段 `instantaneous_fixed_stream_sum_gibps` 应读作“四条固定画像的名义需求和”，不能理解为平滑的瞬时到达。每请求 8 层时，数据量与理想计算时间均乘 8，比值不变。

$(T - N)/C$ 的单位是 token/s，乘 1,408 后是 byte/s；这是**带宽需求，不是时间**。无竞争、40 GiB/s 的纯带宽 SSD 模型下，自身服务时间为：

$$S_i = \frac{V_i}{40} \times 1000 \quad (\text{ms}).$$

真正决定是否停顿的是从释放读取到 HBM ready 的总延迟：

$$L_i = W_{\text{before}} + S_i + W_{\text{between}} + T_{\text{tail}}.$$

分别表示：首次服务前等待、自身服务、自身命令之间的等待、最后 SSD 完成后的接收尾部。相邻层同为固定计算时间 C_i 时：

$$\text{barrier}_i = \max(0, L_i - C_i).$$

SSD 与接收链路可以按命令流水重叠，不能再把整层 $V/50$ 一律串行加在 SSD 完成时间后； T_{tail} 只计最后残留的接收时间。

3.3 预取怎样形成反馈

本层计算开始时提交下一层读取，本层计算结束就是下一层数据的 deadline。用 x_k 表示第 k 层计算开始、 F_{k+1} 表示下一层 HBM ready，则：

$$x_{k+1} = \max(x_k + C_k, F_{k+1}).$$

数据晚到会推迟下一层计算，也推迟再下一层读取。最后一层开始计算时，预取下一个已到达请求的 Layer 0；仍只提供一个计算窗口，不是无限深预取。

4 输入怎样算出来，以及怎样找到候选

4.1 四条输入：拆成两张表

沿用仓库习惯，1K = 1,024 token。“命令”指本模拟器的 QoS I/O 命令，不是 NAND 物理页。

NPU	总 token T	NQL N	SSD token $T - N$	命令/层
0	1,024	169	855	7
1	196,608	368	196,240	1,534
2	196,608	896	195,712	1,529
3	196,608	896	195,712	1,529

NPU	KV/层 (GiB)	计算/层 (ms)	名义需求 (GiB/s)
0	0.001121163	0.582149	1.925903
1	0.257329941	12.390165	20.768888
2	0.256637573	29.856563	8.595684
3	0.256637573	29.856563	8.595684

$$1.925903 + 20.768888 + 8.595684 + 8.595684 \simeq 39.886158 \text{ GiB/s.}$$

NPU0 是**短计算、小读取、怕排队**的流；NPU1 是**大读取、较短计算**的流；NPU2/3 是**大读取、较长计算**的流。这些角色名称不代表额外调度优先级。

四条输入均不是原表的离散点：NPU1/2/3 在 192K 长度上沿 NQL 插值；NPU0 还把 32K/48K 的长度关系外推至 1K。文件哈希能证明来源未变，不能证明原表是硬件实测。

TTFT 别混用：输入画像中的“78 层等效 TTFT”是 $78C$ ，而本例 8 层的理想纯计算总时间是 $8C$ 。例如 NPU0 分别为 45.407660 ms 与 4.657196 ms，均不是含 I/O 停顿的实际请求延迟。模拟器 SLO 统计的请求耗时从 admission 到 completion，不含所有请求在时间 0 到达后等待接纳的时间。

下面用原表与四则运算分别算 NPU0、NPU1，不必先背表里的数字。

4.2 手算 NPU0：1K 总长度，NQL=169

第一步：算 SSD 上的历史 KV。

$$\begin{aligned} T - N &= 1024 - 169 = 855 \text{ token}, \\ V_0 &= 855 \times 1408 = 1,203,840 \text{ byte}, \\ V_0/2^{30} &= 0.001121163368 \text{ GiB}. \end{aligned}$$

每条满命令含 128 token，所以 $855 = 6 \times 128 + 87$ ：6 条满命令加 1 条尾命令，共 7 条。

第二步：从原表取得计算时间端点。单位为微秒 μs 。

总长度	NQL=128 的计算/层 (μs)	NQL=256 的计算/层 (μs)
32K	1178.025959	1997.479696
48K	1513.570276	2668.568314

在每个 NQL 上，沿 32K 与 48K 两点确定的直线外推到 1K：

$$C_{1K,n} = C_{32K,n} + \frac{1 - 32}{48 - 32} (C_{48K,n} - C_{32K,n}).$$

得到 $C_{1K,128} = 527.908845 \mu\text{s}$, $C_{1K,256} = 697.245498 \mu\text{s}$ 。

第三步：在 NQL=128 与 256 之间插值到 169。

$$a = \frac{169 - 128}{256 - 128} = \frac{41}{128} = 0.3203125,$$

$$\begin{aligned} C_0 &= 527.908845 + a(697.245498 - 527.908845) \\ &\simeq 582.149492 \mu\text{s} = 0.582149492 \text{ ms}. \end{aligned}$$

这就是 **0.582 ms 的来源**：它是外推/插值结果，不是“169 token 在任意 NPU 上一定算这么久”。

第四步：分别算名义带宽与自身读取时间。

$$B_0 = \frac{0.001121163368}{0.000582149492} \simeq 1.925903 \text{ GiB/s},$$

$$S_0 = \frac{0.001121163368}{40} \times 1000 = 0.028029084 \text{ ms}.$$

自身读取约 0.028 ms，计算预算 0.582 ms；单独服务时很宽裕。但共享 Path0 的排队可以远大于这两者。

4.3 手算 NPU1 : 192K 总长度, NQL=368

第一步：算数据量。

$$\begin{aligned}T - N &= 192 \times 1024 - 368 = 196,240 \text{ token}, \\V_1 &= 196,240 \times 1408 = 276,305,920 \text{ byte}, \\V_1/2^{30} &= 0.257329940796 \text{ GiB}.\end{aligned}$$

$196,240 = 1533 \times 128 + 16$, 即 1,533 条满命令加 1 条尾命令。

第二步：直接在原表的 192K 行插值, 无需长度外推。

原表点	每层计算时间 (μs)
(192K, NQL=256)	8708.365858
(192K, NQL=512)	17123.906302

$$b = \frac{368 - 256}{512 - 256} = \frac{112}{256} = 0.4375,$$

$$\begin{aligned}C_1 &= 8708.365858 + b(17123.906302 - 8708.365858) \\&\simeq 12390.164802 \mu\text{s} = 12.390164802 \text{ ms}.\end{aligned}$$

第三步：算带宽与自身读取时间。

$$B_1 = \frac{0.257329940796}{0.012390164802} \simeq 20.768888 \text{ GiB/s},$$

$$S_1 = \frac{0.257329940796}{40} \times 1000 = 6.433248520 \text{ ms}.$$

为什么 NQL 只有 NPU0 的 2.18 倍, 计算时间却是 21.28 倍? 因为同时把总长度从 1K 改成了 192K。原表的计算时间同时依赖总长度与 NQL; 两次计算不处于相同上下文。

这解释的是**本表的数值生成方法**, 不能单凭比例证明真实 kernel 的规律。尤其 NPU0 是远离最短 32K 数据点的外推, 短序列固定开销、算子与并行方式可能改变结果。

NPU2/3 同理: 在 (192K,512) 和 (192K,1024) 之间, 用权重 $(896 - 512)/512 = 0.75$ 插值, 得到 29.856563 ms; 每层自身 SSD 服务约 6.415939 ms。

4.4 为什么选两条“长计算的大流”

这不是低利用率的必要条件，也不是必须保住两张卡满载才能构造坏输入，只是本次可解释的搜索策略：

- NPU2/3 每层各读约 0.2566 GiB，足以制造大突发；29.8566 ms 的计算窗口又能隐藏本次 trace 中各自的读取。
- 因为几乎不等待，它们的节拍保持稳定，周期性地向队列注入大量命令。
- NPU0 的计算窗口只有 0.582 ms，最怕这种长队列；NPU1 进一步增加竞争。

长计算降低的是**该流的长期名义需求**，提高的是**它自己**掩盖读取的能力；每次释放的数据量仍然很大。

两张卡始终满载还会使四卡平均值至少为 50%。若探索更低的全局平均利用率，就不应把这个角色组合固定成必要前提。

4.5 连续输入与有限搜索

四个 lane 分别预置 3,000、150、150、150 个请求，原实验均在时间 0 到达。这是充足 backlog 的闭环压力测试，不是真实流量回放。短流数量更多，是为了防止它先耗尽任务。

最后固定 NPU1/2/3，只对 NPU0 的 NQL=153...192 统一复跑，选出 169。NQL=153 的名义和更高 (39.9962 GiB/s)，平均利用率反而为 76.4921%，高于 169 的 76.2261%。**更靠近带宽上限，不保证利用率更低。**

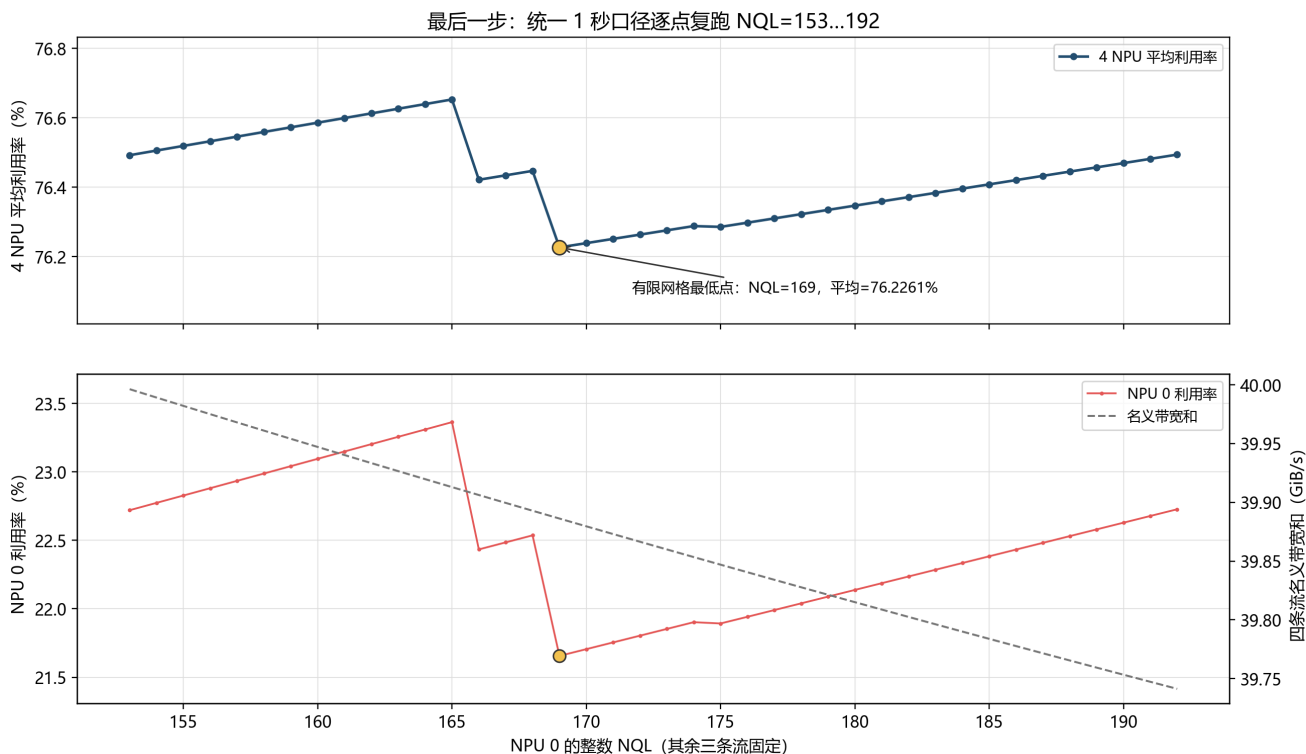


图 2：有限搜索；跳变反映事件次序与相位关系，不是简单的带宽单调函数。

trace 记录了 207,297 条与窗口相交的 SSD 命令：全部进入 Path0，入队与服务次序失配为 0，每 SSU 同时最多服务一条命令。观察器开关前后的仿真摘要哈希相同，说明采集没有改变本次运行。

5 为什么等待造成错位，仍没消除低利用率

5.1 错位是事实，自动变均匀不是必然

你的推理前半段成立：某层等过以后，下一层读取会推迟。缺失的一步是：**推迟后的时刻，是否一定避开其它流下一次突发？**

本例没有独立的外部计时器持续均匀释放 I/O，而是在计算开始时提交下一层读取。等待改变下一次到达，到达又进入有 backlog 的同一 FIFO，后续等待继续受它控制。

为什么“等待导致错位”不等于“以后不会再撞车”

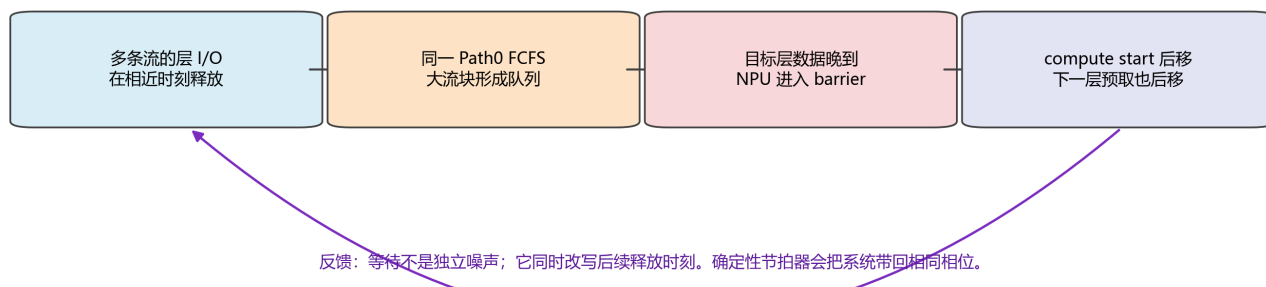


图 3：等待改变计算开始，计算开始又决定下一层 I/O 释放。

在本次确定性 trace 中，改变后的相位最终重复。准确说法是**观察到周期性释放模式**，不是证明所有初始状态都会收敛到同一周期。

5.2 用一个周期看清楚

下图放大一秒窗口的前 100 ms，时间均相对窗口起点。

Baseline 单 Path0 的计算、等待和物理服务

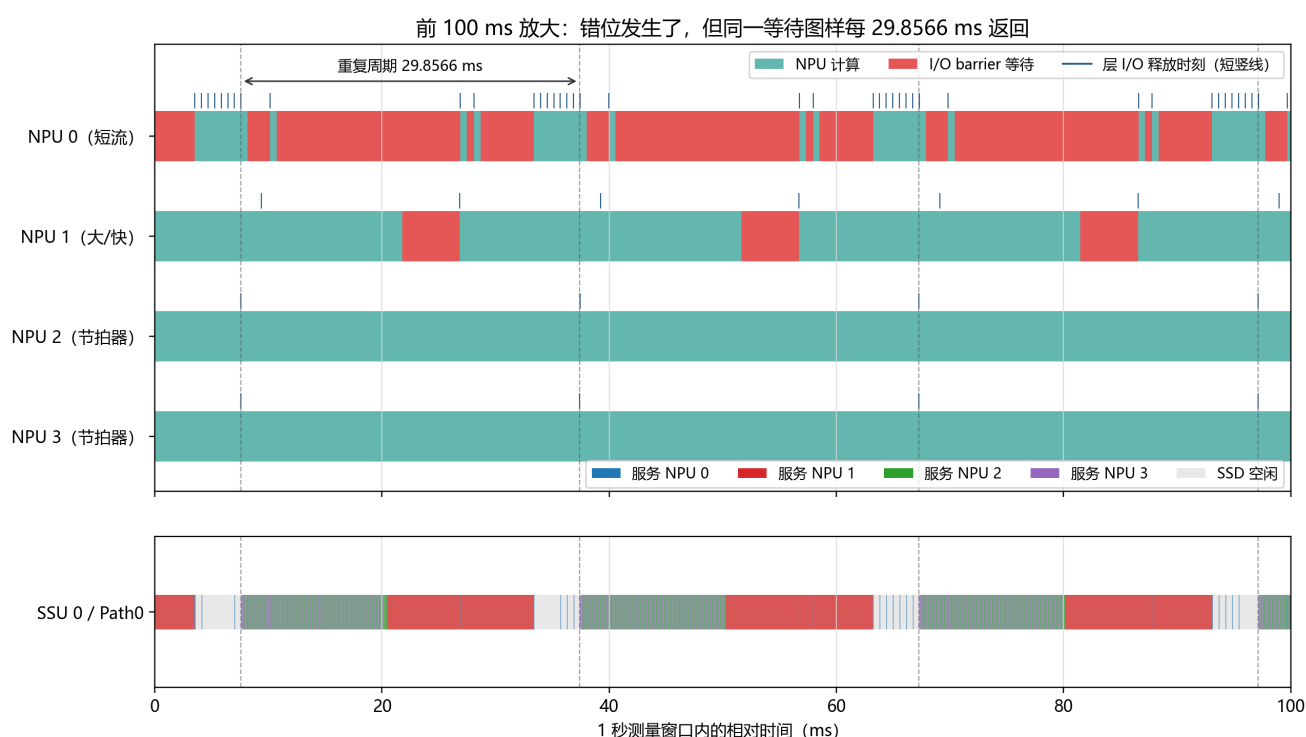


图 4：读取释放、NPU 计算/等待和 SSD 实际服务的联合时间线。

第一个完整周期中：

1. 约 7.557/7.562 ms, NPU3/2 各释放约 0.2566 GiB。
2. 7.589 ms, NPU0 释放仅 7 条命令的小层, 到约 10.131 ms 才 ready, 出现约 1.959 ms 等待。
3. NPU0 到 10.131 ms 才计算并释放再下一层；新读取到约 26.881 ms 才 ready, 又等待约 16.169 ms。
4. NPU1 在 9.387 ms 释放的大层也先经历排队, 产生约 5.076 ms 等待。
5. 约 37.414 ms, NPU3 下一轮突发到来, 结构再次出现。

令 $P = 29.856562816$ ms。稳态 trace 中可见：

$$\begin{aligned} \text{NPU2/3: } 1 \times 29.856562816 &= P, \\ \text{NPU1: } 2 \times 12.390164802 + 5.076233212 &= P, \\ \text{NPU0: } 11 \times 0.582149492 + 23.452918408 &= P. \end{aligned}$$

即同一周期内, 有的卡算一层, 有的算两层, 有的算十一层并夹杂长等待。它们不必每层同时开始, 仍能周期性重现相同队列局面。

相位没有持续漂移：散点形成竖列, 说明释放模式被锁在固定相位
(若等待能不断把流打散, 竖列会逐周期向左右移动或扩散)

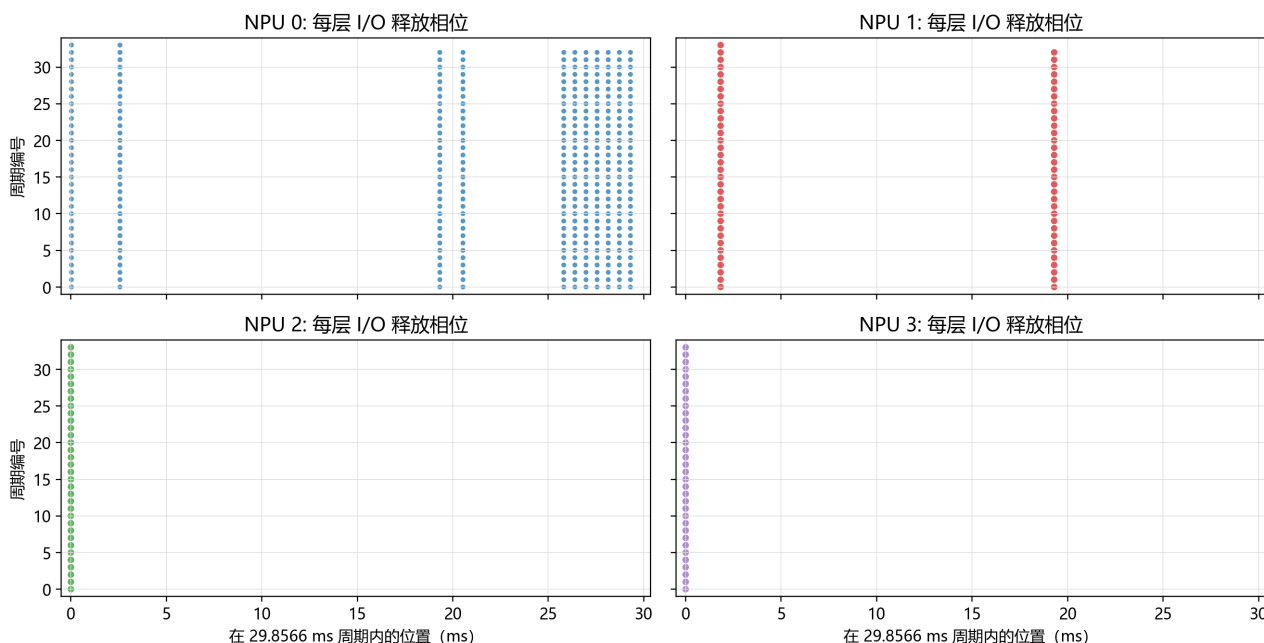


图 5：将释放时刻折叠到周期内；竖列说明本次运行的相位稳定，而非不断漂移。

5.3 跨请求预取已经开启，为何还不够

NPU0 只提前约 0.582 ms 发出读取, 坏情况下某些命令首次服务前就等约 16.7 ms。跨请求预取既不提高优先级, 也不越过现有队列。

所以不是没有预取, 而是**提前量远小于排队尾延迟**。第 10 节会计算加深预取的时间与空间成本, 不能笼统说“多预取两层即可”。

6 归因看完整时序，不能只看等待时 SSD 在读谁

6.1 NPU1：排队早发生，停顿晚暴露

取 request_id=100020, layer=3, 时间相对测量窗口起点：

事件	时刻 (ms)
释放该层读取	9.387382
第一条命令开始 SSD 服务	20.417279
本层数据的 deadline	21.777547
最后一条命令完成 SSD 服务	26.850528
数据到 HBM, 层 I/O ready	26.853780

从释放到 ready 的分解是：

首次服务前等待 = 11.029897 ms,
自身 1534 条命令连续服务 = 6.433249 ms,
接收尾部 = 0.003252 ms,
合计 = 17.466398 ms.

预算只有 12.390165 ms, 因此停顿为：

$$17.466398 - 12.390165 = 5.076233 \text{ ms.}$$

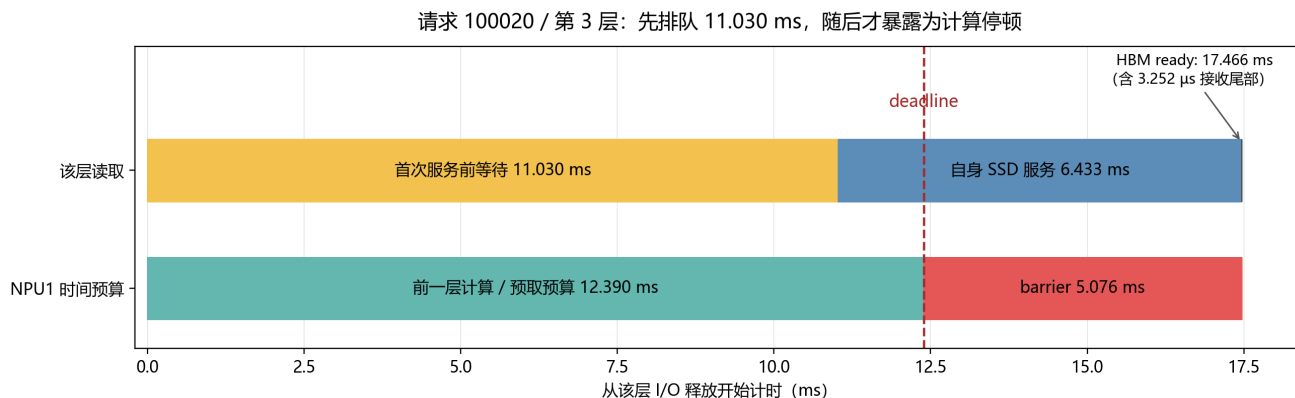


图 6: deadline 落在自身服务中，不代表之前的排队与停顿无关。

关键纠正： barrier 暴露期间，SSD 确实主要读 NPU1 自己的数据；但自身服务原本只需 6.433 ms, 可以被 12.390 ms 的计算隐藏。此前约 11.030 ms 的排队推迟了服务起点，最终突破 deadline。

“等待时 SSD 正在服务谁”只是**时间重叠统计**，不能直接当成因果分摊。应同时检查 release、首次服务、最后服务、ready 和 deadline，必要时对服务顺序做干预对照。

6.2 NPU0 的长尾有多严重

NPU0 的 783.440 ms barrier 中，779.590 ms 与其它 NPU 的 SSD 服务重叠，3.756 ms 与自身服务重叠，剩余约 0.095 ms 是空闲或接收尾部。结合 FIFO 顺序与四 Path 对照，这支持“小而急的请求被前方其它流命令拖住”的解释。

其**命令级**排队中位数仅 0.0205 ms, P95 却为 16.7323 ms, P99/最大值约 16.7445 ms。不能把命令百分位直接当成层延迟百分位；一层要等多条命令全部到齐。

另按“deadline 落在中间一秒内”的层统计：

NPU	层数	超时层数	超时比例	最大 release→ready (ms)
0	372	134	36.02%	16.750784
1	67	33	49.25%	17.466398
2	34	0	0%	12.859069
3	34	0	0%	12.687026

NPU0 的超时比例不及 NPU1，但坏等待相对其 0.582 ms 计算窗口更长，时间利用率反而更低。**超时次数、超时严重度与整卡利用率要分开看。**

6.3 为什么 NPU 在等，SSD 却能空闲

39.8862 GiB/s 是按无等待计算节拍得到的名义需求。发生 barrier 后，下一层读取释放也推迟，实际到达速率随之下降。

按完整锁相周期估算，四条流约产生 0.4131、17.2377、8.5957、8.5957 GiB/s，合计 34.8422 GiB/s；实际一秒内 SSD 服务吞吐为 34.7642 GiB/s。窗口边界与队列存量变化使两者不必完全相等。

这形成“排队导致计算停顿，停顿又减少未来 I/O 到达”的反馈。名义需求不超过 40 GiB/s，既不是逐层延迟保证，也不排除 SSD 空闲。

7 对照与敏感性：不只是一组精确周期

7.1 只改变 Path 映射

保持同一输入、物理 SSU、40 GiB/s、静态 QoS 表与预取方式，只把四个 NPU 分到 Path0/1/2/3。调度器可在四个队头之间仲裁，但后端带宽没有增加。

配置	平均利用率	NPU0	NPU1
Baseline：全部 Path0	76.2261%	21.6560%	83.2484%
四个独立 Path 对照	95.8124%	100.0000%	83.2495%

两者 NPU2/3 均为 100%。各按相同预热规则测量一秒，窗口起点分别为 2430.127253 与 2431.080242 ms，不是强行使用同一启动瞬间。

该对照支持**模型内因果判断**：单 FIFO 缺少流间隔离是 NPU0 低利用率的关键条件。对照不是 Baseline 搜索结果，也不证明多 Path 能满足每条流的所有 deadline。

NPU1 基本没改善，只说明**这一次具体跨 Path 仲裁仍没解决它的停顿**，不能反推原先那 11 ms 排队不重要。共享带宽、仲裁份额、突发次序与预取窗口仍共同决定 ready 时刻。

7.2 五组已复跑的确定性测试

全部保持单 Path0、8 层、跨请求预取、中间一秒统计与名义和不超过 40 GiB/s；所有仿真不变量通过。只改指明的变量。

测试	名义和 (GiB/s)	平均利用率	NPU0	NPU1
原参数, seed=42	39.8862	76.2261%	21.6560%	83.2484%
仅改 seed=1	39.8862	76.2261%	21.6560%	83.2484%
启动错开	39.8862	78.3173%	15.1526%	98.1166%
NPU3 NQL=900	39.8480	78.5778%	14.5537%	99.7573%
NPU3 NQL=928	39.5898	78.2365%	15.7763%	97.1696%

NPU2/3 在所有测试中都为 100%。具体扰动：

- 启动错开：lane 到达起点为 0、7.5、15、22.5 ms，每 lane 内仍有充足 backlog，不是持续随机到达抖动。
- NPU3 NQL=900：计算由 29.8566 改为 29.9892 ms。
- NPU3 NQL=928：计算改为 30.9176 ms。改 NQL 同时更新 KV 数据量与带宽，不是单改时钟。

已支持：不要求所有 lane 同时启动，也不要求两条长流的计算时间完全相等，当前模型仍可低于 80%。

尚不支持：这些点不能给出真实负载中的出现概率，未测试每层随机 jitter、真实设备并行、HBM 容量限制或完整 78 层模型。

注意：启动错开后均值变好，NPU0 却从 21.66% 降到 15.15%。改善均值不等于改善短流公平性。

7.3 是否只是窗口碰巧截到坏时段

原参数的一秒切成十个 100 ms 子窗口，平均利用率在 75.0176%–77.3054% 之间。敏感性测试的逐块范围也存入结果文件。这支持窗口内持续低利用率，不证明所有更长时间与其它输入都相同。

8 现实中的 SSD→HBM Prefill 会出现吗

8.1 机制可能出现，76.23% 不能照搬

在无主机 DRAM 数据缓存场景中，下列条件若同时满足，长尾排队导致低计算利用率是合理风险：

1. 多 NPU 共享同一 SSU 的有限服务能力。
2. 某些层读取很大，短时间集中进入不可越过的队列。
3. 其它层计算短、预取窗口短，deadline 对排队敏感。
4. 数据晚到时，没有其它已 ready 的计算可调度到该卡。
5. 读取释放依赖计算开始，等待反馈到未来到达。

这些条件**不需要主机 DRAM 参与，也不需要原参数的精确锁相**。随机性可能打散固定周期，仍留下概率性的长尾碰撞。出现频率与严重度要由真实输入和硬件 trace 证明。

8.2 先核实真实 Path 是否等于本模型 FIFO

NPU2/3 每层各 1,529 条命令，客户端每 0.1 μ s 提交一条，约 **0.1528 ms** 发完整层；后端单独消化却需 **6.4159 ms**。快速提交与慢速消化形成长队列。

模型还采用每 SSU 一条不可抢占活动命令，未限制 Path 的 QD（在途/排队命令数），也未模拟真实介质延迟。因此它能展示 FIFO 风险，不能自动代表 flash 内部执行方式。

NVMe 规范允许普通命令在同一或不同 Submission Queue 中重排处理，也不要求按提交顺序完成（融合操作等另有约束）。所以**一个 NVMe 队列不天然等于本项目的严格 FIFO Path0**。实际 SSU 的 FIFO 约束究竟在软件入口、QoS 调度器还是数据可用之前，需单独确认。[S2：NVMe Base 2.0a, §3.1.2](#)

介质并行也不保证消除问题：前端若仍把紧急命令挡在长队列后，延迟仍可能增加。但必须用实际 QD、仲裁与并行度验证。

8.3 GLM-5.1 的读取量与计算量还需落地

官方配置含 78 层、前 3 层 dense、index_topk=2048，最大位置数为 202,752。本例 192K = 196,608 token 在其范围内，但只运行 8 层且每层计算时间相同，并未模拟 dense/MoE 层差异。[S3：GLM-5.1 官方配置](#)

1,408 byte 可对应一种 BF16 打包假设 $(512 + 64 + 128) \times 2$ ，但**维度不能单独证明实际 SSD 格式就是这个大小**。应核实 KV 布局、索引是否分开、压缩量化、对齐过读与每卡分片。

DSA 按 query 选择 token 索引；不同实现可能直接使用稀疏索引，也可能先生成掩码。不能因 top-k=2048 就把整个 Prefill 层的 SSD 读取量定为 2,048 token：多个 query 的选择并集、索引读取、块粒度过读及是否预取全前缀都会影响实际字节数。[S4：Transformers GLM DSA 实现](#)

无主机 DRAM 时，**哪些 KV 已在 HBM、哪些必须从 SSD 取、HBM 有多少预取空间**仍是变量。但本次结果没有凭空引入 HBM 命中率，仍按构造的每层 SSD 字节数执行。

“500 TFLOP/s”只能给出指定 FLOPs 模型下的理想算时下界。精度、kernel、稀疏实现、通信、内存带宽与固定开销会改变实测 C 。不要用未经验证的理想时间替换表中时间后，就称其为硬件预测。

8.4 应采哪些实机数据

至少用相同的层/请求 ID 对齐：

- 实际 compute start/end，及是否有其它请求填补等待。
- 实际 SSD 字节、命令大小，而不仅是逻辑 token 长度。
- Path queued bytes、active remaining bytes、QD 与服务次序。
- release、首次服务、SSD 最后完成、**HBM ready** 与 deadline。

同时检查链路拓扑，以及直通路径是否发生兼容回退。这不是建议引入 DRAM，而是防止实测违背“无 DRAM”的实验条件。[S1：直通与兼容路径说明](#)

9 写一个判断 Path 是否赶得上 deadline 的函数

9.1 冻结的单 FIFO 可以按字节快速估算

设 Q 为活动命令**剩余**字节，加上所有确定排在目标前面的 queued bytes； R 为目标还需读取的字节。若目标全部命令位置已确定、只有该 Path 竞争、后端为纯带宽模型，则：

$$T_{\text{ssd}} = \frac{Q + R}{40 \times 2^{30}} \times 1000 \text{ ms}, \quad \text{slack} = D - T_{\text{ssd}}.$$

D 为剩余预算。slack 非负表示预计满足，数值为提前量；负值表示预计不能满足，绝对值为欠缺时间。目标已经入队时，不要把其字节同时计入 Q 和 R 。

以 NPU0 为例，忽略接收尾部，能容忍的前方 backlog 为：

$$\begin{aligned} Q_{\text{max}} &= 40 \times \frac{0.582149492}{1000} - 0.001121163368 \\ &= 0.0221648163 \text{ GiB} \simeq 22.697 \text{ MiB}. \end{aligned}$$

约 132 条 176 KiB 满命令，而一条大流的一层有 1,529 条。22.697 MiB 是这个冻结队列和 deadline 下的预算，**不是全系统通用的安全 QD**。

9.2 只给 I/O 个数够不够

不够，除非等大且大小已知。100 条 176 KiB 命令的数据量是 100 条 4 KiB 命令的 **44 倍**。

BLOCK_SIZE=128 指 **128 token/命令**，不是“128 个 KV block”，也不是 128 KiB：

$$128 \times 1408 = 180,224 \text{ byte} = 176 \text{ KiB}.$$

要分清应用 KV block、主机/QoS I/O 命令、SSD 内部 NAND 页。4 KiB 逻辑块或对齐要求，不意味着 NAND 颗粒只支持 4 KiB；一次主机读取可覆盖多个逻辑块，大小受控制器和软件栈限制。若真要聚合 128 个 KV block，先给出每 block 的 token 数，再核实硬件命令上限。[S2：NVMe Base 的命令与传输限制](#)

本模型的尾命令未强制 4 KiB 补齐。真实直通若要求对齐，实际读取量可能大于逻辑 KV 字节，需将补齐与过读计入。

9.3 IOPS、固定延迟与多 Path 的边界

带宽和 IOPS 同时限制时，简化吞吐耗时下界为：

$$T_{\text{throughput}} = \max \left(\frac{Q + R}{B}, \frac{N_Q + N_R}{I_{\text{max}}} \right).$$

式中时间为秒， B 为 byte/s。这是聚合估算，不是任意并行 SSD 的精确完成时间。固定开销、介质延迟、QD、并发和尾延迟仍需考虑；IOPS 若已包含同一开销，不能再重复叠加。

40 GiB/s 若全是 4 KiB 命令，需 **10,485,760 IOPS** 才能跑满；若全为 176 KiB 满命令，约需 **238,313 IOPS**，纯带宽服务约 4.196 μs /命令。客户端 0.1 μs 提交间隔不是 SSD IOPS 参数。

9.4 两个可以直接使用的例子

例 A：给定等大小命令数，并考虑 IOPS 上限。

```
from path_deadline_estimator import (
    estimate_uniform_fcfs_path_deadline,
)

result = estimate_uniform_fcfs_path_deadline(
    path_io_ahead=100,
    required_io_count=10,
    io_size_bytes=4096,
    time_budget_ms=1.0,
    bandwidth_gib_s=40.0,
    iops_limit=100_000,
)
print(result.meets_deadline)  # False
print(result.shortfall_ms)   # 约 0.1 ms
```

这里 IOPS 下界 $110/100000 = 1.1$ ms，超出 1 ms 预算。函数预测 **SSD 阶段**，HBM ready 还需接收尾部。目标尚未全部入队、有未来命令交错，或多 Path 共享服务时，应使用完整事件模型；不能无条件把设备峰值当成每条 Path 的保证带宽。

例 B：直接用前方字节数判断 NPU0 是否赶得上。

```
from path_deadline_estimator import estimate_fcfs_path_deadline

result = estimate_fcfs_path_deadline(
    path_bytes_ahead=256 * 2**20,  # 256 MiB ahead
    required_bytes=1_203_840,      # NPU0 layer KV bytes
    time_budget_ms=0.582149492,
    bandwidth_gib_s=40.0,
)
```

SSD 阶段预计 $6.25 + 0.028029 = 6.278029$ ms，欠缺约 **5.695880 ms**。这个例子没有设置 IOPS ceiling：即使纯带宽足够高，前方字节量也能让短 deadline 失败。若目标命令已经分散在队列中，必须按其实际相对位置计算，不能把整个 Path 的所有字节一律当作前方字节。

10 无 DRAM 前提下怎样优化系统

以下是工程建议，不是已完成性能测试。优先级按本次证据排列，应同时关心均值、短流 deadline 与公平性。

10.1 优先减少紧急读取前方的排队字节

可选方向：独立可仲裁 Path；按 deadline/字节份额调度；大突发分批提交，限制一条流一次排在别人前面的字节量。

四 Path 对照已显示更换可选队头能改善 NPU0。仅缩小单条命令，却仍一次性把同一批全排在 FIFO 前面，未必减少小流前方的总字节。

代价：更多队列状态、调度和遥测；小命令增加 IOPS/提交成本；过度节流会减少设备并行和吞吐。上节的 132 条不能当作未经验证的全局配置。

10.2 深预取必须计算时间与 HBM 空间

旧版“提前两层或更多层就能吸收 10-17 ms”过于含糊。NPU0 的预算为：

$$2C_0 = 1.164299 \text{ ms}, \quad \left\lceil \frac{16.7}{0.582149492} \right\rceil = 29.$$

若坏排队仍为 16.7 ms 量级，需要约 **29 个这样的连续计算窗口**的提前量，两层明显不足；严格覆盖 17 ms 则需 30 个。

但每请求只有 8 层，现有跨请求预取仅取下一请求 Layer 0。这意味着更深的跨请求/跨层机制，并要求未来 KV 地址已知；若稀疏选择依赖当前 query 计算结果，过早精确预取未必可行。

29 份 NPU0 每层 KV 约 **33.29 MiB**；这只是该流 KV 数量级，未含当前工作集、索引、对齐和其它请求。对约 0.2566 GiB/层的大流同样保留 29 份，则约 **7.44 GiB**。无 DRAM 时，缓冲必须占用 HBM，挤占权重、活跃 KV 和计算工作区。

提前更多读取还会改变队列本身。“29”是保持原延迟不变的预算估算，不是实现后消除等待的保证；应同时限制预取在途字节并重跑 trace。

10.3 错峰与调度其它计算

可以显式安排大读取的发送节拍，但不能把它推过自己的 deadline。敏感性结果说明，简单错开启动可能改善均值，却进一步伤害短流。

若允许某请求等 I/O 时执行另一个 HBM 数据已 ready 的请求，整卡空闲可能减少。这不需要 DRAM，但需要 HBM 空间、独立可运行工作与调度器支持，会改变当前 batch=1 的执行语义。

不要用无限增加某类请求排队时间来换取利用率。一起报告逐卡利用率、层超时比例/超时量、请求实际延迟和 HBM 峰值占用。

10.4 推荐验证顺序

先校准实际计算与 SSD 字节，再加入 QD、介质并行和接收链路；之后分别测试队列隔离、按字节限突发、受 HBM 预算约束的深预取。每次只改一个可解释条件，保留相同一秒计量口径。

目前已验证的优化对照只有四 Path 映射；其它建议均待测，不能把预测写成收益承诺。

11 复现、证据与来源

11.1 文件与复现命令

原始结果、输入表、搜索表和 trace 未修改。新版另存为带 _v2 的 Markdown/PDF；旧文档保留，并在 revision_archive/ 留有备份。

构建脚本会调用独立复核器，重算输入、覆盖时间、层超时与 NPU1 时序，并检查来源哈希。完整测试与数字审计保存在结果目录的 tutorial_sensitivity_results.json 和 tutorial_numeric_audit.json。

在仓库根目录运行，不重跑原搜索：

```
python run_baseline_path0_tutorial_sensitivity.py --jobs 3
python -m unittest -v test_path_deadline_estimator.py
python build_baseline_path0_tutorial.py
```

需要项目 Python/绘图依赖、Pandoc、XeLaTeX 与文档指定字体。PDF 可用 pdfinfo、pdftoppm 检查。

从头重跑原搜索和四 Path 对照，建议用新目录保留既有证据：

```
python run_baseline_4npu_ssul_low_utilization.py \
  --output results/baseline_path0_reproduction
```

先读 input_profiles.csv 与 search_summary.csv，再用 request_layer_timeline.csv 对齐层的 release/ready/deadline，用 physical_block_trace.csv 查入队与服务。旧文件名中的 physical block **不表示真实 NAND 页操作**。

实现入口：

- policy_logic.py: baseline_path_ids() 决定 Path0。
- sim.py: PathQueue 实现 FIFO。
- continuous_batch_sim.py 中的两个预取触发函数：
_handle_compute_schedule();
_start_cross_request_layer0_prefetch()。

11.2 外部来源与支持范围

核对日期为 2026-09-05。外部资料支持架构边界，不是 76.2261% 的来源；该数值来自本地仿真与 trace。

1. [S1: NVIDIA GDS Overview Guide](#)：直接 DMA、控制面与数据面、兼容回退；不是本项目 NPU/SSU 规格。
2. [S2: NVMe Base Specification 2.0a](#)：命令次序与传输限制；不定义本项目自有 QoS Path。
3. [S3: GLM-5.1 官方配置](#)：层数、索引、维度与最大位置数；不提供对应硬件的计算时间。
4. [S4: Transformers GLM DSA 实现](#)：query 级 top-k 和实现差异；不定义你的 SSD KV 格式。

11.3 最终判断

已验证：当前单 FIFO、固定画像、有限预取的 Baseline 可在名义和不超过 40 GiB/s 时，使中间一秒平均计算时间利用率低于 80%；五个确定性测试均成立。

机制：短计算流容忍的前方排队字节很少；大突发推迟其读取，等待又反馈到下一层释放。错位不保证消除后续碰撞。原参数形成稳定周期，但原周期并非低利用率的必要条件。

现实预测：无主机 DRAM 数据缓存的 SSD→HBM Prefill 仍可能受到这一机制影响；具体数值取决于实际调度、读取量、计算时间、HBM 缓冲、设备并行和可调度工作。未经校准，不把仿真百分比当成硬件结论。